methods

CSCI 134

certain operations
are frequently applied to lists

```
[5, 1, 2, 5, 0, 7]
```

how many times does a particular value appear?

e.g. 5 appears twice

where is the first appearance of a particular value?

e.g. 0 appears at index 4

we could write functions to perform these operations

how many times does a particular value appear?

```
def count(ls, value):
    result = 0
    for val in ls:
        if val == value:
            result = result + 1
    return result
```

where is the first appearance of a particular value?

```
def index(ls, value):
    for i in range(len(ls)):
        if ls[i] == value:
            return i
    return -1
```

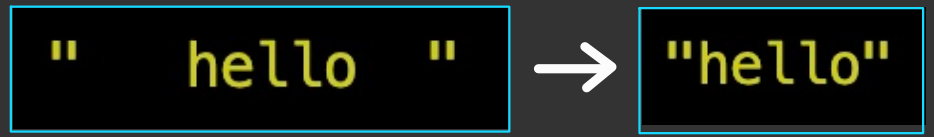
but python already provides them!

```
In [1]: nums = [5, 1, 2, 5, 0, 7]

In [2]: list.count(nums, 5)
Out[2]: 2

In [3]: list.index(nums, 0)
Out[3]: 4
```

certain operations

are frequently applied to strings

strip leading and trailing **whitespace**



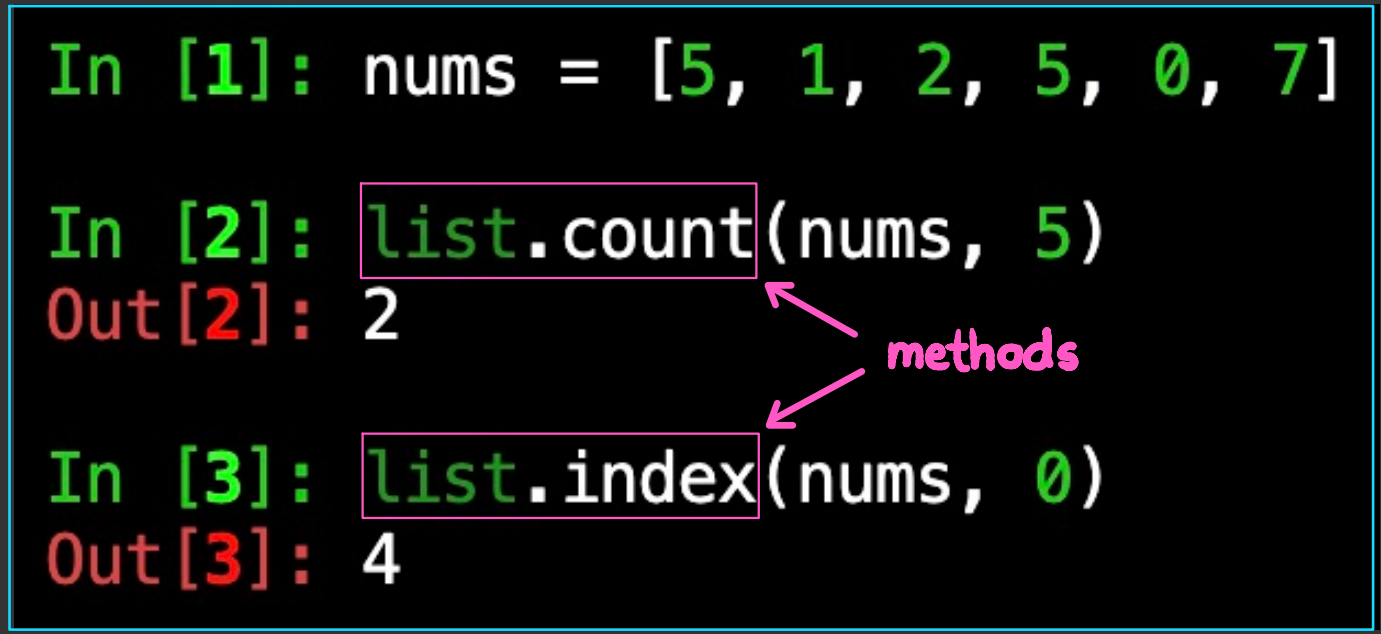
split a string into components

python provides these too!

```
In [1]: str.strip("   hello  ")
Out[1]: 'hello'

In [2]: str.split("do,re,mi", ",")
Out[2]: ['do', 're', 'mi']
```

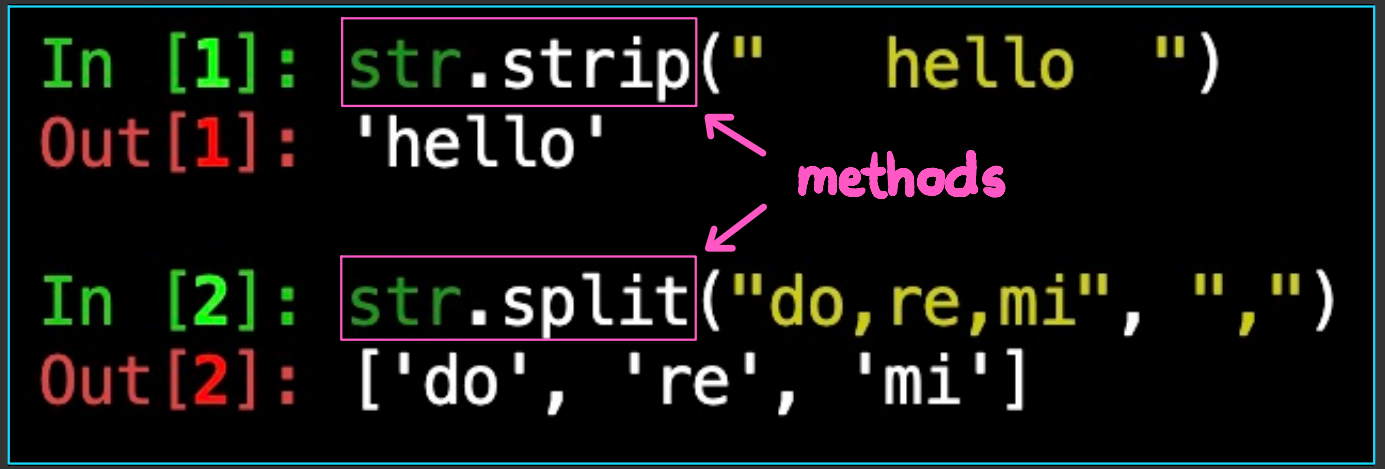
Functions associated with a type

are called methods

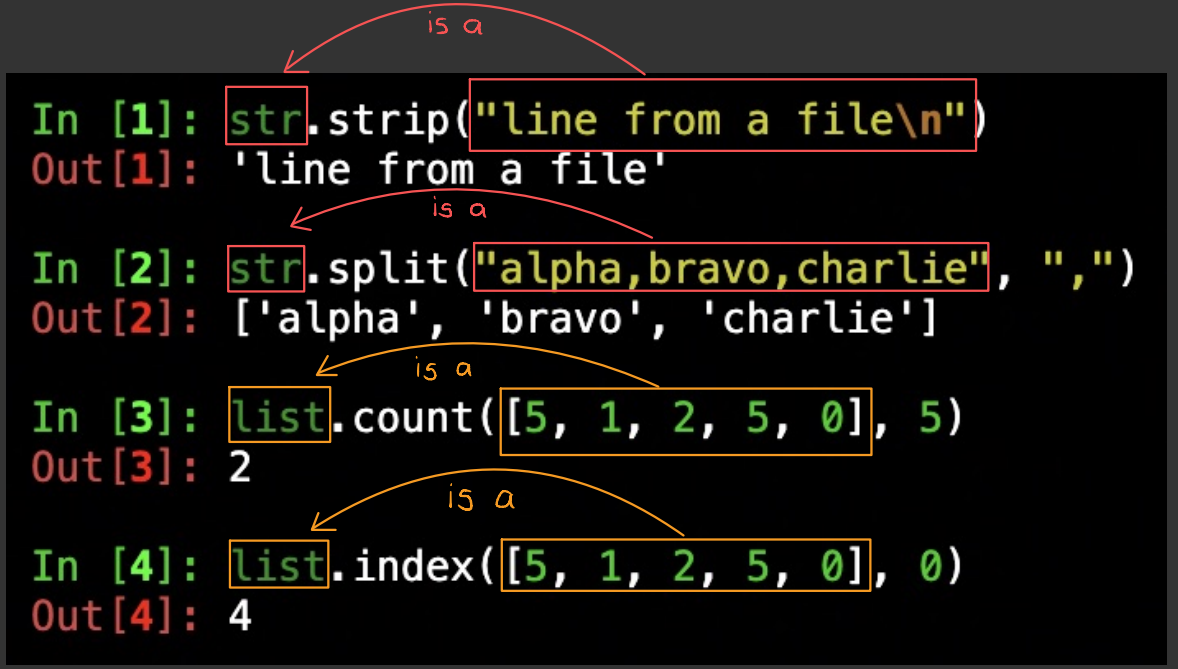


Functions associated with a type

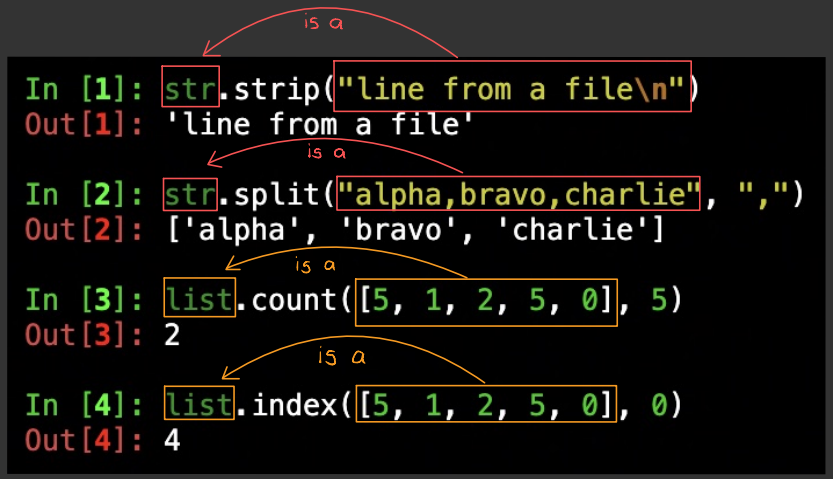
are called methods



the First argument of a method is always an expression of that type



this permits a convenient shorthand




```
In [1]: "line from a file\n".strip()
Out[1]: 'line from a file'

In [2]: "alpha,bravo,charlie".split(",")
Out[2]: ['alpha', 'bravo', 'charlie']

In [3]: [5, 1, 2, 5, 0].count(5)
Out[3]: 2

In [4]: [5, 1, 2, 5, 0].index(0)
Out[4]: 4
```

this is such a common shorthand that many programmers are unaware of the longhand

`type . method ( arg 1 , arg 2 , ... , arg n )`

```
In [1]: str.strip("line from a file\n")
Out[1]: 'line from a file'

In [2]: str.split("alpha,bravo,charlie", ",")
Out[2]: ['alpha', 'bravo', 'charlie']

In [3]: list.count([5, 1, 2, 5, 0], 5)
Out[3]: 2

In [4]: list.index([5, 1, 2, 5, 0], 0)
Out[4]: 4
```

`arg 1 . method ( arg 2 , ... , arg n )`

```
In [1]: "line from a file\n".strip()
Out[1]: 'line from a file'

In [2]: "alpha,bravo,charlie".split(",")
Out[2]: ['alpha', 'bravo', 'charlie']

In [3]: [5, 1, 2, 5, 0].count(5)
Out[3]: 2

In [4]: [5, 1, 2, 5, 0].index(0)
Out[4]: 4
```

but it can be helpful to know that a lot of python is just convenient shorthand for

methods

```
In [1]: nums = [3, 1, 4, 1, 5, 9]

In [2]: len(nums)
Out[2]: 6

In [3]: list.__len__(nums)
Out[3]: 6

In [4]: 4 in nums
Out[4]: True

In [5]: list.__contains__(nums, 4)
Out[5]: True
```

some useful str methods

```
In [1]: school = "Williams College"

In [2]: school.upper()
Out[2]: 'WILLIAMS COLLEGE'

In [3]: school.lower()
Out[3]: 'williams college'

In [4]: school.split()
Out[4]: ['Williams', 'College']

In [5]: school.index("m")
Out[5]: 6

In [6]: "***".join(["a", "b", "c"])
Out[6]: 'a***b***c'

In [7]: ", ".join(["a", "b", "c"])
Out[7]: 'a, b, c'

In [8]: "line from a file\n".strip()
Out[8]: 'line from a file'
```

some useful list methods

```
In [1]: letters = ["a", "b", "c"]

In [2]: letters.append("b")

In [3]: letters
Out[3]: ['a', 'b', 'c', 'b']

In [4]: letters.count("b")
Out[4]: 2

In [5]: letters.index("a")
Out[5]: 0

In [6]: letters.extend(["d", "e", "f"])

In [7]: letters
Out[7]: ['a', 'b', 'c', 'b', 'd', 'e', 'f']
```